BÀI TẬP 2

Tích hợp hệ thống & Xử lý lỗi

TIẾP NHẬN VÀ PHÂN LOẠI YÊU CẦU BẢO HÀNH

Sinh viên: Nguyễn Gia Huy

MSSV: 2374802010181

Lớp: [BỔ SUNG MÃ LỚP]

Track: Software Engineering (SE)

Repository: github.com/NguyenGiaHuy1608/cdtn1-nguyengiahuy-l2

Phiên bản dự kiến: v0.2

LƯU Ý: Trước khi nộp, bổ sung ảnh chụp chạy thật vào các khung minh chứng ở Mục 3, 4 và 5. Không nộp bản còn khung chờ ảnh.

Mục 1. Tóm tắt hệ thống

Ứng dụng hỗ trợ tiếp nhận yêu cầu bảo hành điện thoại: tra cứu/tạo khách hàng, chọn thiết bị và nhóm lỗi, tạo phiếu bảo hành, xem danh sách và chi tiết phiếu. Dữ liệu được lưu trong SQLite.

Phạm vi chính

• Khách hàng: đọc, tạo, cập nhật và xóa theo ràng buộc dữ liệu.

• Phiếu bảo hành: tạo, xem danh sách/chi tiết, cập nhật mô tả và ưu tiên, xóa phiếu.

• Danh mục: thiết bị có hãng/model/serial riêng; tình trạng bảo hành được trình bày riêng, không dùng thay tên thiết bị.

• Xác thực demo bằng bearer token; API nghiệp vụ yêu cầu token.

• Pipeline dữ liệu mẫu và mô hình phân loại mô tả lỗi được cung cấp như thành phần học thuật riêng.

Công nghệ

Python 3 · Flask · SQLite · HTML/CSS/JavaScript thuần · unittest · Python standard library cho pipeline và mô hình cơ sở.

Ngoài phạm vi

Chưa triển khai phân công kỹ thuật viên, sửa chữa, thanh toán, thông báo tự động hoặc quyết định bảo hành tự động dựa trên AI. Mô hình chỉ minh họa phân loại mô tả lỗi.

Mục 2. Kiến trúc đã hiện thực

Luồng phụ thuộc được tổ chức theo lớp, nhằm tách giao tiếp HTTP khỏi quy tắc nghiệp vụ và truy cập dữ liệu.

Lớp	Trách nhiệm	Ví dụ
Presentation / Routes	Nhận request, kiểm tra token, trả JSON và mã HTTP.	src/routes/api.py
Service	Thực thi luồng nghiệp vụ và phối hợp repository/validator.	src/services/
Repository / DAO	Truy vấn và ghi dữ liệu; đóng/mở kết nối SQLite.	src/repositories/
Database	Lưu khách hàng, thiết bị, nhóm lỗi, phiếu và lịch sử trạng thái.	migrations/, seeds/, data/
Frontend	Hiển thị màn hình và trạng thái; gọi HTTP qua API client riêng.	templates/, static/api/

Hai nhóm CRUD tiêu biểu

• Customer API: GET/POST/PUT/DELETE /api/customers; xóa bị từ chối nếu khách hàng còn thiết bị liên kết.

• Ticket API: GET/POST/PUT/DELETE /api/tickets; áp dụng kiểm tra dữ liệu, trạng thái và quy tắc bảo hành.

Tách HTTP client ra khỏi controller frontend: static/api/client.js và các module theo miền nghiệp vụ; static/app.js tập trung trạng thái giao diện/render.

Mục 3. Module cốt lõi — minh chứng chạy được

Chạy từ thư mục gốc sau khi cài requirements. Ảnh dưới đây phải được thay bằng ảnh thật chụp từ máy sinh viên, thể hiện lệnh đã chạy và kết quả.

Lệnh khởi động

```
python app.py
```

DÁN ẢNH CHỤP THẬT TẠI ĐÂY

Ảnh terminal có Flask khởi động thành công và/hoặc trình duyệt hiển thị ứng dụng. Nên thấy URL 127.0.0.1:5000 và màn hình ứng dụng thực tế.

Ảnh này dùng cho Mục 3 của PDF. Không chụp code tĩnh thay cho chương trình đang chạy.

Mục 4. Bộ kiểm thử — minh chứng PASS

Bộ kiểm thử hiện có kiểm thử API/CRUD và kiểm thử pipeline/mô hình. Có ca luồng đúng, dữ liệu biên và dữ liệu sai.

Lệnh chạy toàn bộ

```
python -m unittest discover -s tests -v
```

Lệnh pipeline và AI

```
python scripts/data_pipeline.py
python scripts/train_baseline.py
```

DÁN ẢNH CHỤP THẬT TẠI ĐÂY

Ảnh terminal sau khi chạy unittest; phải thấy số test đã chạy và dòng OK. Chụp riêng ảnh pipeline/model nếu ảnh đầu không đủ chỗ.

Không ghi kết quả PASS nếu terminal báo FAILED/ERROR. Nếu thiếu Flask, cài bằng python -m pip install -r requirements.txt rồi chạy lại.

Mục 5. Kết quả — ba trạng thái giao diện

Cần minh chứng ba trạng thái loading, empty và error của các vùng tải dữ liệu. Chỉ sử dụng ảnh chụp thật của ứng dụng. Nếu khó tái hiện empty/error, ghi rõ cách tái hiện trong chú thích.

ẢNH 1 — LOADING

Chụp khi vùng dữ liệu đang tải. Tái hiện bằng cách làm mới trang hoặc throttling mạng trong DevTools nếu cần.

ẢNH 2 — EMPTY

Chụp vùng dữ liệu không có bản ghi. Có thể dùng database test rỗng hoặc tài khoản/test data phù hợp; không xóa database dữ liệu thật.

ẢNH 3 — ERROR

Chụp trạng thái lỗi và nút thử lại; tái hiện bằng cách tạm ngắt API hoặc dùng môi trường test. Khôi phục server sau khi chụp.

Mục 6. Cách dựng lại hệ thống

Thực hiện từ thư mục gốc repo trên Windows PowerShell.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
python app.py
```

Mở http://127.0.0.1:5000. Tài khoản demo mặc định: demo / demo123. Không sử dụng secret mặc định trong môi trường thật. File .env và database cục bộ bị loại khỏi Git qua .gitignore.

Chạy pipeline và huấn luyện mô hình

```
python scripts/data_pipeline.py
python scripts/train_baseline.py
```

Pipeline có ba pha: load CSV, validate với năm nhóm kiểm tra, chuẩn hóa và xuất CSV sạch. Mô hình Multinomial Naive Bayes là baseline demo, huấn luyện từ dữ liệu mẫu nhỏ, không dùng để quyết định tự động việc chấp nhận/từ chối bảo hành.

Thay đổi so với BT1

• Bổ sung luồng CRUD và kiểm thử tích hợp API.

• Bổ sung tên hãng/model thiết bị và giữ trạng thái bảo hành ở cột riêng.

• Bổ sung pipeline dữ liệu, mô hình phân loại baseline, tài liệu chạy và khai báo AI.

• Giữ SQLite và vanilla JavaScript để giảm chi phí cài đặt; đánh đổi là cần tự quản lý migration/schema và test UI thủ công.

Mục 7. Phụ lục — khai báo sử dụng công cụ AI

Công cụ: ChatGPT. Phạm vi hỗ trợ: rà soát cấu trúc phân lớp, đề xuất kiểm thử, soạn tài liệu, pipeline dữ liệu và mô hình baseline.

Cách kiểm chứng

- Đọc lại và hiểu các thay đổi trong mã nguồn; đối chiếu với phạm vi đề tài.
- Chạy unittest, pipeline và script huấn luyện trên máy thực tế.
- Chụp ảnh minh chứng từ terminal/trình duyệt thật; không dùng ảnh dựng.
- Cập nhật nội dung khai báo nếu quá trình sử dụng AI thực tế khác bảng này.

Tôi xác nhận sẽ đọc, hiểu, kiểm chứng và chịu trách nhiệm với mã nguồn và nội dung nộp. Các kết quả PASS chỉ được ghi nhận sau khi thực sự chạy thành công.

Nguyễn Gia Huy · MSSV 2374802010181

TRƯỚC KHI NỘP

1. Bổ sung mã lớp ở trang bìa.
2. Thay ba khung chờ ở Mục 3, 4, 5 bằng ảnh chụp thật.
3. Chạy lại toàn bộ test sau khi cập nhật.
4. Tạo tag Git v0.2 và xác nhận repo Public, không chứa .env/secret/database cá nhân.